Algorithms Selected

Uninformed – 1. Breadth-First Search (BFS)
2. Uninformed Cost Search (UCS)

Informed – 1. Greedy Best First Search
2. A^*

GROUP 17

NAME	ID
Prachi Garg	2023A7PS0548P
Jyotsna Lal	2023A7PS0660P
Soham Kudva	2023A7PS0589P
Daksh Tyagi	2023A7PS0647P

Problem Parameters

State:

$$S = (x, y, C, E)$$

Where;
1. $(x, y) \longrightarrow$ robot position
2. $C \subseteq I, \ |I| = k \longrightarrow$ collected items
3. $E \in \{0, \ldots, E_{max}\} \longrightarrow$ remaining energy

State Space Size

Valid positions: $N = (\text{width} \times \text{height}) - O$

Total states: $|S| \leq N \cdot 2^k \cdot (E_{max} + 1)$

Branching Factor b

Actions: move up, down, left and right.

So, $b \leq 4$

Depth definitions

d = optimal solution's depth

m = maximum depth

$m \leq |S|$

$d \leq E_{max}$

Since each move costs 1 energy

Greedy Best-First Search (GBFS)

Evaluation Function

Greedy Best-First Search expands the node with the smallest heuristic value.

$$f(n) = h(n)$$

Where $h(n)$ is either:

- Manhattan-to-delivery:

$$h_1(n) = |x - x_d| + |y - y_d|$$

- Landmark (ALT-style) heuristic:

$$h_2(n) = \max_{l \in L} |dist(l, D) - dist(l, n)|$$

Greedy does not consider path $g(n)$. In this, each action consumes 1 unit of energy, and a node is considered feasible only if remaining energy $e(n) \geq 0$. Nodes with $h_1(n) \geq e(n)$ are pruned since robot can't reach delivery location with remaining energy.

Time Complexity

Step 1: Branching Factor

For the warehouse grid: 1. Possible moves: Up, Down, Left, Right

2. Maximum branching factor: $b \leq 4$.

Step 2: Maximum Depth of State Space

State representation: $S = (x, y, C, E)$

Total no. of states $\Rightarrow |S| \leq N^2 \cdot 2^k \cdot (E_{max}+1)$

maximum possible depth $\Rightarrow m = O(N^2 \cdot 2^k \cdot E_{max})$

Step 3: Worst Case Expansion

Greedy selects nodes based solely on heuristic value.

If the heuristic provides poor guidance (e.g. ignores uncollected items), the algorithm may explore deep, non-optimal branches before finding a goal.

Even with energy pruning, in the worst case Greedy may generate the entire feasible search tree up to depth m: $1 + b + b^2 + \dots + b^m$

This geometric series gives: $O(b^m)$

Since $b \leq 4$, time complexity is $O(4^m)$

Substituting m: $O(4^{N^2 \cdot 2^k \cdot E_{max}})$

Thus, Greedy has exponential ~~difference~~ worst-case time complexity, although energy constraints may reduce the practical search space.

Space Complexity

Greedy maintains:

- A priority queue (frontier) ordered by $h(n)$.
- An explored set (in graph search version)

In the worst case, it may generate all reachable feasible states.

Therefore:

$$\text{Space Complexity} = O(b^m) = O(4^m) = O(4^{N^2 \cdot 2^k \cdot E_{max}})$$

Space Complexity is also exponential.

Completeness

Greedy's completeness depends upon implementation.

Tree Search Version: If no repeated-state checking is used, the algorithm may revisit states in cycles. In this case, it is not complete.

Graph Search Version (with closed list): In this problem the state space is finite: $N^2 \cdot 2^k \cdot (E_{max}+1)$. If repeated states are avoided, every feasible state can be explored at most once. Energy constraints ensure that paths exceeding available energy are pruned, so the search is restricted to feasible states only. Therefore, Greedy is complete in finite state spaces when implemented as graph search. Since the warehouse grid, item combinations, and energy levels are finite, Greedy is complete under graph search.

Optimality

GBFS is not optimal.

Reason: It selects nodes based only on estimated remaining distance $h(n)$. It ignores path cost already incurred $g(n)$. In the warehouse problem: The robot may move toward the delivery location because it has small heuristic value. However, some items may remain uncollected far from the delivery zone. This can cause unnecessary backtracking and increased total path length, even if the solution satisfies the energy constraint. Since greedy does not minimize total path cost, it doesn't guarantee minimum-step solutions. Thus:

Greedy Best First Search is not optimal.

(II) Informed Search :- A* Algorithm Analysis

We apply A* on the state space: $s = ((x,y), C, e)$ where (x,y) is robot position, C is collected item subset and e is remaining energy. Each move costs 1 energy, obstacles are static & transitions are deterministic

1) Time complexity :- Analyses done in different phases, based on the steps done by the robot.

Step 1 :- From any state robot can attempt 4 moves (up, down, left, right) so worst case b = branching factor = 4 so $\boxed{b = O(4)}$

Step 2 :- d = depth = Number of actions in the shortest valid plan from start to goal state i.e reaching delivery cell D, while still feasible with energy. so $\boxed{d <= E_{max} \text{ otherwise no solution exists}}$

Step 3 :- m = maximum depth A* can expand to, since each action takes 1 energy ~~and energy~~ so $\boxed{m = E_{max}}$

Step 4 :- Worst case; A* may explore all nodes upto depth d so $O(b^d)$ here $(b <= 4)$ $O(4^d)$

$\because$ $d <= m = E_{max}$ so worst case upper bound :- $O(4^{E_{max}})$

Step 5 : We have a finite graph here so Total possible states:

$$|S| <= N * 2^k (E_{max} + 1)$$

here N = Number of free cells (<= RC)
k = Number of items
$E_{max}+1$ energy levels
2^k subset of collected items

[Each set is expanded at most once with consistent heuristic]

so time is $\boxed{O(|S| \log |S|)}$ log used $\because$ A* maintains a priority queue, each push/pop is $O(\log |open|)$ & $|open| <= |S|$

$\therefore$ Time complexity = $O(|S| \log |S|)$

2) Space Complexity :-

A* stores the following :-

1) OPEN (frontier) priority queue
2) CLOSED (explored) set of visited states
3) Parent Pointers / g - cost arrays for path reconstruction

Note :- ∵ we use Pareto dominance pruning here it reduces the No. of stored labels per state in practice but the theoretical worst case space complexity remains exponential in No. of items

worst case: OPEN + CLOSED can contain almost all reachable states

so, space $= O(|S|) = O(N * 2^K * E_{max})$ which is why A* here is very memory heavy as K grows.

3) Completeness :- This answers if it will find a solution if one exists.

A* is complete under the following conditions :-

1. Finite branching factor (true here ∵ $b <= 4$)
2. Step costs are positive (true here ∵ as mentioned each move costs 1 energy)
3. The search space is finite (true here due to energy restrictions & closed set)

here in our problem,

Max depth $= m = E_{max}$ = no infinite paths

Also Number of states is finite so if a feasible path to delivery exists with energy budget, A* will find it

so $\boxed{\text{Yes, A* here is complete}}$

4) Optimality :- Answers, will it find the best solution!

so A* is optimal (returns a min cost path) if:

1) h(s) is admissible (never overestimates true remaining cost)
2) For graph search, consistency (monotonicity) ensures no re-expansions

and guarantees optimality clearly.

In our setup :-

1. Path cost is energy used (No. of steps)
2. Uses an admissible and consistent heuristic that lower bounds the remaining energy req. to reach delivery location

$\boxed{\therefore \text{ A* is OPTIMAL}}$

Although we use Pareto dominance pruning to reduce expansions as mentioned earlier, so the worst case asymptotic Time complexity remains unchanged.

Note ↑

BREADTH - FIRST SEARCH (BFS)

Breadth first search is a foundational uninformed search algorithm that explores a graph or grid level by level. In context of intelligent warehouse robot, BFS starts at robot's initial configuration (x,y) and expands all possible adjacent moves (up, down, left, right) before moving to states that are 2 steps away

Time complexity :-

The time taken by the algorithm is proportional to the number of nodes expanded in state-space search tree

Derivation : The branching factor b is 4 because in a 2D grid, the robot can move in 4 directions (Up, Down, Left, Right). The depth of the solution d is significantly larger than the physical distance because the robot must visit multiple item location (K) and potentially charging stations before reaching the goal

State space factor: Each step in the search tree now involves checking energy levels and inventory. The effective no of states, the algorithm may visit is $O(N \cdot 2^K \cdot E_{max})$, where N is grid size, K is no of items and E_{max} is energy capacity

Conclusion: Time complexity is $O(b^d)$. For BFS, this grows exponentially.

Space complexity :-

BFS must store every node it reaches in memory to maintain frontier and explored set

Frontier state : It stores all nodes at current depth d that are waiting to be expanded.

→ stores width of search tree at depth d.

For BFS, space complexity is $O(b^d)$ for frontier state. The no of nodes increases exponentially with respect to depth d.

Explored set : It must store every unique tuple (x, y, C, E) that the algorithm has expanded to prevent cycles. In L3, robot can reach same coordinates with different energies, & set of items.

Space complexity is : $O(N * 2^K * E_{max})$

Conclusion : Space complexity is $O(b^d)$

completeness :-

Analysis : BFS is complete.

Proof : Since the branching factor b is finite (4 direction) ~~and path cost is uniform~~ ; as long as there exists at least one sequence of actions that collects the required items and reaches the delivery zone without exhausting energy E , BFS will find it by systematically exploring every possible sequence.

optimality :

Analysis : BFS is optimal only for the number of steps

Justification BFS expands the shallow nodes first. it will find path with fewest actions. It will ignore energy efficiency and item wights and choose the shortest path even if it consumes more power than a slightly longer, more efficient route

BFS guarantees the physically shortest path in a grid and ensures that a solution is found in a finite warehouse space, regardless of obstacle complexity.

Uniform Cost Search

UCS is an uniformed search algorithm used by our agent to navigate the $N \times N$ warehouse grid ($10 \leq N \leq 50$).

UCS treats the energy consumption (1 unit per move) as the path cost $g(n)$

Time complexity :

$b \equiv$ Branching Factor Cost of optimal path : C^*

b is equal to four (representing North, South, East & West)

Due to the collected items being included in our state, the search space expands significantly.

Analysis: UCS expands nodes in order of their cumulative cost. It will explore all states where the energy consumed $g(n)$ is less than the optimal cost C^*.

Worst case time complexity : $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$

$\varepsilon = 1$ energy unit.

Completeness :

→ Status : Complete

→ UCS is guarranteed to find a path to the delivery depot if the solution exists.

→ Conditions : $b = u$ (finite) $\varepsilon = 1$ energy unit.
Because 1 is greater than 0, the algo will never get stuck in an infinite path of zero-cost edges.

→ Energy constraints: If the required energy is greater than the agent's initial battery capacity, the algo will return "No solution".

Optimality:

→ Status: Optimal.

→ Proof: UCS expands nodes in strictly increasing order of their path cost $g(n)$. In our warehouse, the first time the "goal Test" (Items in Depot with the Robot) is satisfied for a node selected for expansion, it is guarranteed to be the path that consumed the min total energy.

→ Constraint: Energy costs are non-negative

Justification: We have selected UCS as one of our 2 uniformed search algo. because the warehouse Robot Problem is essentially a cost-minimization task (minimizing energy consumption/path length. Unlike BFS, which is optimal if all step costs are equal, UCS remains robust even if we were to introduce variable terrain costs (e.g., heavier items increasing energy consumption per move) in future L4 extensions.

Space Complexity :

→ **Frontier & memory** : To ensure optimality & handle the graph search, UCS maintains a frontier (Priority Queue) of all generated but unexpanded nodes & an Explored set (Hash Map) of all visited states (x, y, items-collected)

→ In the worst case, every node explored during the time complexity phase must be stored in memory to ensure the algo can back-track on find alternative cheaper paths.

→ Space Complexity is proportional to the number of nodes stored matching the time complexity.

$$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$$

Summary Table

Property.	UCS
Search type	Uninformed.
Time complexity	$O(b^{1+\lfloor c^*/1 \rfloor})$
Space complexity	$O(b^{1+\lfloor c^*/1 \rfloor})$
Complete	Yes
Optimal	Yes

SUMMARY TABLE

Algorithm	Time complexity	Space complexity	Complete	Optimal
BFS	$O(N \cdot 2^k \cdot E_{max})$	$O(\lvert S \rvert) = O(N \cdot 2^k \cdot E_{max})$	Yes	No
UCS	$O(b^{\wedge}(1 + \lfloor C^*/E \rfloor))$ → $O(N^* 2^k * E_{max})$ * $\log(N^* 2^k * E_{max})$ (Problem specific)	" $O(N^* 2^k * E_{max})$ (Problem specific)	Yes	Yes
Greedy BFS	$O(4^{N^2 \cdot 2^k \cdot E_{max}})$	$O(4^{N^2 \cdot 2^k \cdot E_{max}})$	Yes (Graph search version) No (Tree search version)	No
A^*	$O((N \cdot 2^k \cdot E_{max}) \cdot \log(N \cdot 2^k \cdot E_{max})$	$O(N \cdot 2^k \cdot E_{max})$	Yes	Yes

Among BFS, UCS, Greedy and A^*, we choose A^* as the best algorithm for our problem statement because it combines optimality with efficient search guidance. While BFS & UCS are complete and optimal they explore many unnecessary states since they do not use heuristics. Greedy search is faster but does not guarantee optimal solutions. A^* uses an admissible heuristic to reduce expansions while still guaranteeing the minimum-energy solution making it most suitable algo for warehouse navigation problem.